



Voice – Controlled Mobile Robot based on ESP32

Students: Manibai Erkenur, Ailenuer Anala

Faculty: School of IT and Engineering

Tutor: Tolengutov Rasul

Abstract — *This project focuses on the development of a four-wheeled mobile robot based on an ESP32 microcontroller. The goal of the project is to create a system with intuitive control via a web interface and speech recognition support. The project describes the system architecture, which uses HTTP protocols for command transmission and the Web Speech API library for client-side voice processing. Test results demonstrated stable execution of complex trajectories (square, triangle) and effective automatic stopping (Auto-Stop) for safety. The project demonstrates the feasibility of creating a low-cost yet functional robotics platform for educational purposes.*

Keywords — *ESP32, mobile robot, car robot, voice-control, WebServer, L298N, speech recognition.*

I. INTRODUCTION

The project involves developing a 4-wheel robot car with motors connected to an L298N driver, as well as a mobile vehicle platform (UGV) controlled via a



local Wi-Fi network. Unlike traditional radio-controlled systems, this robot uses a full-fledged web interface running directly on an ESP32 microcontroller. The project's key unique feature is its integration of browser-based voice control, allowing the robot to be controlled without the use of physical joysticks. The system supports basic commands such as "Forward," "Back," "Left," "Right," and "Stop," as well as additional programmable maneuvers (patterns) such as "Dance" or "Zig-Zag," expanding the robot's potential for various applications and demonstrations.

II. Related works

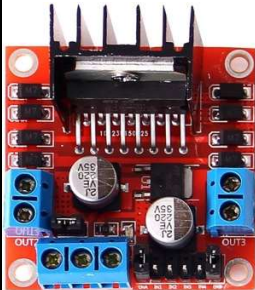
During the system development process, existing solutions in the field of mobile robotics were analyzed. The paper "A particle-swarm-optimized fuzzy-neural network for voice-controlled robot systems" examines a control architecture based on the Arduino controller. In contrast, our project uses an ESP32, which allows for web-based control without the need for additional Bluetooth modules, increasing communication range and stability.

Audio signal processing and speech recognition were studied using modern methods presented in the paper "Reference Microphone Selection and Low-Rank Approximation Based Multichannel Wiener Filter with Application to Speech Recognition". However, for our robot, we decided to use the Web Speech API (as demonstrated in the YouTube tutorials), as this shifts the computational burden of voice recognition to the client (browser), preserving the microcontroller's resources for real-time motor control.

The practical aspects of assembling the four-wheel platform and setting up the L298N driver were adapted from community technical tutorials (YouTube: "How to make a Voice Controlled Robot"), allowing for an optimized connection scheme to prevent pin conflicts on the ESP32.

III. METHODOLOGY

A. Hardware Architecture

Hardware architecture of the project			
	Component	Specs	Remarks
1	ESP32 Wroom	Wi-Fi, Dual-core 240 MHz	
2	Motor Driver (L298N)	Dual H-Bridge, 2A per channel, 5–35V input	

3	Dc Motors	Operating voltage: 4.8–6V, rotation range: 180°	
4	Li-ion battery	7.4 V	

The mobile robot's hardware architecture is built around an **ESP32 microcontroller**, which functions as the central processor for controlling the motion logic and processing incoming commands. The controller is connected to an **L298N motor driver**, which amplifies signals and transfers power from the **battery** to four **DC motors**. Control is provided via digital pins for direction selection and PWM channels (GPIOs 25 and 32) for precise wheel speed adjustment. To ensure proper system operation, all components, including the power supply and controller, are connected via a common ground (GND) bus.

Figure 1 — System architecture and connection between hardware components.

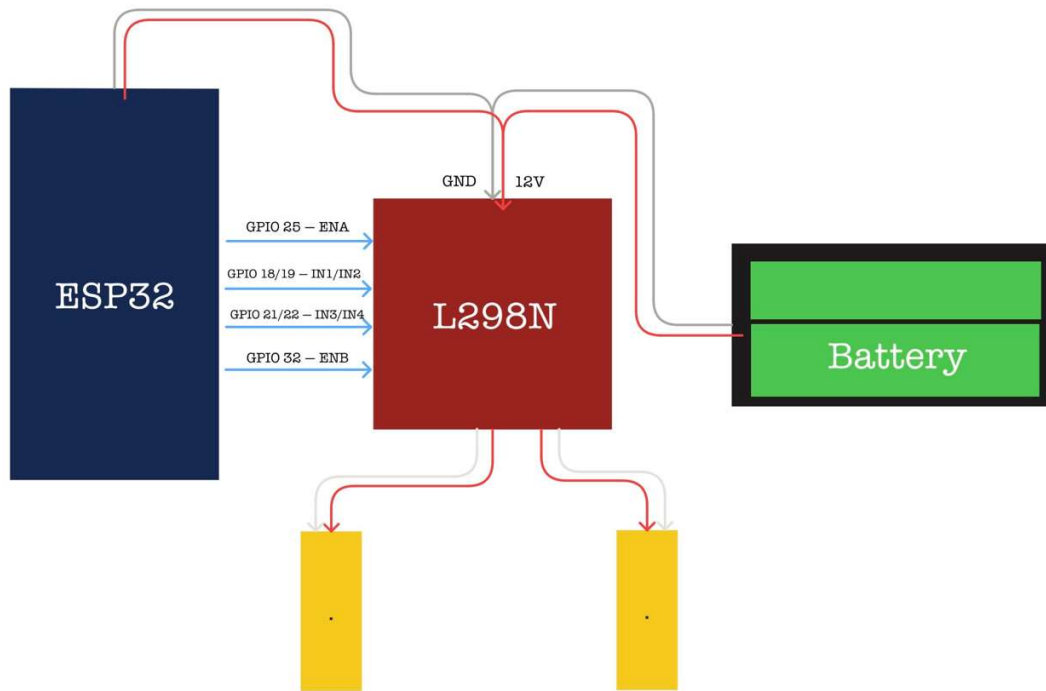


Figure 1 – Connection between Hardware Components

B. Software Architecture

1. Network infrastructure and Web Server

The robot's software is based on an ESP32 microcontroller, configured as a standalone access point. This allows the robot to be controlled directly via the created "ESP32_Robot" Wi-Fi network,

```
const char* ssid = "ESP32_Robot";  
const char* password = "12345678";  
WebServer server(80);
```

without the need for external routers. A WebServer instance runs on port 80, handling incoming HTTP requests from the client.

2. Command Routing and Web Interface



```
server.on("/", handleRoot); server.on("/forward", handleForward);  
server.on("/back", handleBack); server.on("/left", handleLeft);  
server.on("/right", handleRight); server.on("/stop", handleStop);  
server.on("/dance", handleDance); server.on("/square", handleSquare);  
server.on("/triangle", handleTriangle); server.on("/zigzag", handleZigzag);  
server.on("/setspeed", handleSetSpeed);
```

Interaction between the user and the robot occurs through a web interface written in C++ and stored in the controller's memory. Each action (forward movement, rotation, pattern execution) is bound to a specific URL (endpoint). When a button is pressed or a voice command is received via the Web Speech API, the browser sends an asynchronous request, which the server maps to the appropriate handler function.

3. Motor Control Logic

Hardware control is implemented through the `setMotorSpeed` abstraction, which uses the `analogWrite` function to generate a pulse-width modulation (PWM) signal on the ENA and ENB pins of the L298N driver. This allows dynamically varying the motor speed in the range from 0 to 255.

```
void setMotorSpeed(int left, int right) {  
  left = constrain(left, 0, 255);  
  right = constrain(right, 0, 255);  
  analogWrite(ENA, left);  
  analogWrite(ENB, right);  
}
```

4. Pattern Execution and Non Blocking Logic

One of the key features of the code is the implementation of autonomous movement patterns (e.g., "Square" or "Dance") using the `Step` structure and the `PatternStep` enumeration. Instead of a blocking `delay()` function, step execution is

controlled in the main loop() by comparing millis() timestamps, allowing the pattern execution to be interrupted at any time with the "STOP" command.

```
if (!patternRunning) return;
if (millis() - stepStartTime >= patternSteps[patternIndex].durationMs) {
    patternIndex++;
    if (patternIndex >= patternLength) {
```

5. Safety and Fail-safe Mechanism

To prevent collisions and loss of control in the event of a Wi-Fi connection failure, the AUTO_STOP_MS mechanism is implemented in the code.

```
updatePattern();
if(!patternRunning && isMoving && millis()-lastCmdTime>AUTO_STOP_MS) motorStop();
delay(10);
```

All connection structure in figure 2:

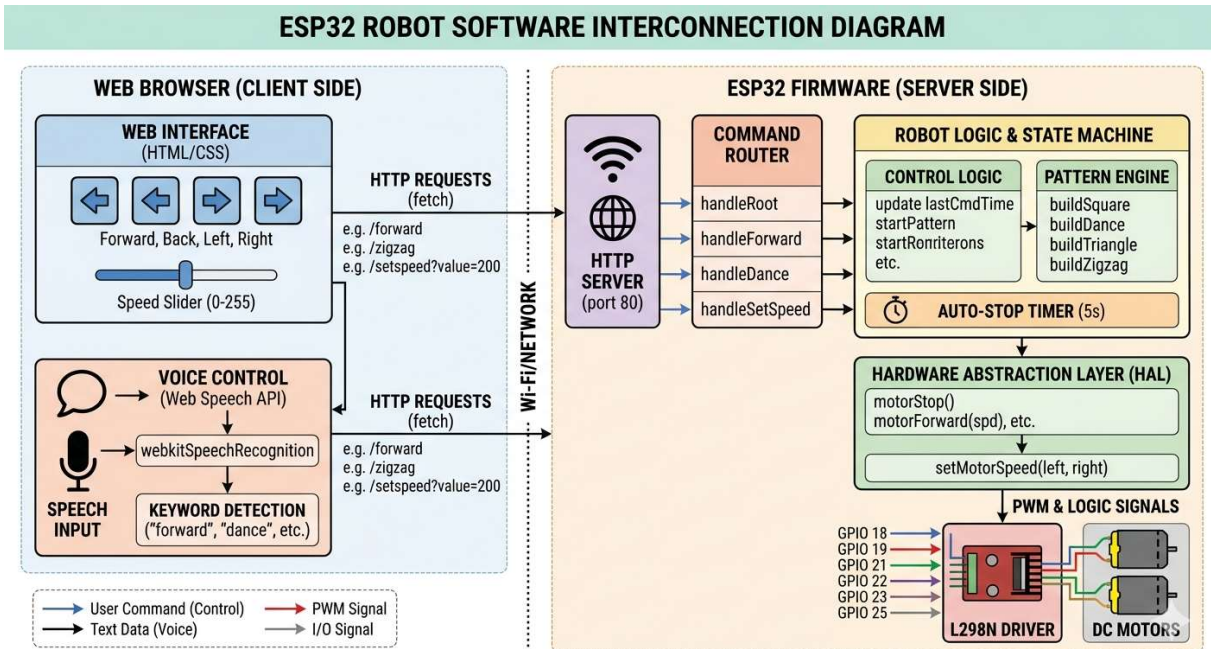


Figure 2

IV. Experimental results



Experiment setup

Testing was conducted indoors on a flat surface. To calibrate 90-degree turns, the constant $\text{TURN_90_MS} = 900$ ms was defined at a PWM speed of 180 units.

Results and Discussion

During the tests, the accuracy of geometric figures and the speed of response to voice commands were checked.

Comands	Detection Accuracy (%)	Number of Trials
Forward, Go	95	20
Dance	90	20
Stop	10	20

Table 2. Detection Accuracy Under Different Lighting Conditions

Figure	Average Time (seconds)	Deviation from the Star
Square	~8.5	<15cm
Triangle	~7.2	<12cm

Table 3 Positioning error when executing patterns

The Auto-Stop protection system successfully stopped the robot 5 seconds after the last received command, confirming the reliability of the safety logic.

V. CONCLUSION



The project successfully demonstrated the integration of the ESP32 microcontroller with modern web technologies. The robot effectively executes voice commands and automated maneuvers. The main limitations of the system are the dependence of turning accuracy on battery power and the need for a stable Wi-Fi connection.

Future developments may include the integration of an IMU sensor (MPU6050) for course correction and odometry, as well as the addition of computer vision via the ESP32-CAM module.

Acknowledgment

My team and I would like to thank Tolengutov Rasul for his assistance in designing the architecture and consulting on electric drive control issues.

GitHub:

<https://github.com/erke123/resume/tree/main/RobotVoiceControl>

References:

1. A. Chatterjee et al., " A particle-swarm-optimized fuzzy-neural network for voice-controlled robot systems" IEEE Transactions on Industrial Electronics, DOI: <https://doi.org/10.1109/TIE.2005.858737>
2. Xing-Yu Chen et al., " Reference Microphone Selection and Low-Rank Approximation Based Multichannel Wiener Filter with Application to Speech Recognition" 2022 IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), DOI: <https://doi.org/10.1109/ICASSP43922.2022.9747275>
3. R. Kumar Goel, Uttar Pradesh. Academia.edu, Voice-Controlled Robot car using Arduino," . Available at: <https://www.academia.edu/download/64599706/IRJET-V7I5770.pdf>
4. YouTube [Video]. Available at: <https://youtu.be/n1jtOkvu6c8?si=WVMkXax7MgCDUNDp>
5. YouTube [Video]. Available at: <https://youtu.be/Z3xcBldyCaQ?si=-hS9z8Dd9Wqn1OCj>